

1.6 IF Statements

Sue has just received her third text message in the last minute. Not only are her best friends text-aholics, but it seems that new people are texting her every day. Sometimes she feels like she is swimming in a sea of text messages. “If only I could filter them like I do my e-mail messages” thinks Sue as four new messages appear on her phone in quick succession. Deciding to put a stop to this madness, she writes a program to filter her text messages. This program features a series of IF statements, each containing a rule to route the messages to the appropriate channel.

Objectives

By the end of this class, you will be able to:

- Create an IF statement to modify program flow.
- Recognize the pitfalls associated with IF statements.

Prerequisites

Before reading this section, please make sure you are able to:

- Declare a Boolean variable (Chapter 1.5).
- Convert a logic problem into a Boolean expression (Chapter 1.5).
- Recite the order of operations (Chapter 1.5)

Overview

IF statements allow the program to choose between two courses of action depending on the result of a Boolean expression. In some cases, the options are “action” and “no action.” In other cases, the options may be “action A” or “action B.” In each of these cases, the IF statement is the tool of choice.

Action/No-Action

The first flavor of the IF statement is represented with the following syntax:

```
if (<Boolean expression>)  
    <body statement>;
```

For example:

```
{  
    if (assignmentLate == true)  
        assignmentGrade = 0;  
}
```

The Boolean expression, also called the controlling expression, determines whether the statements inside the body of the loop are to be executed. If the Boolean expression (`assignmentLate == true` in this case) evaluates to `true`, then control enters the body of the IF statement (`assignmentGrade = 0;`). Otherwise, the body of the IF statement is skipped.

Action-A/Action-B

The second flavor of the IF statement is represented with the following syntax:

```
if (<Boolean expression>)
    <body statement>;
else
    <body statement>;
```

For example:

```
{
    if (grade >= 60)
        cout << "Great job! You passed!\n";
    else
        cout << "I will see you again next semester...\n";
}
```

Much like the Action/No-Action IF statement, the Boolean expression determines whether the first set of statements is executed (`cout << "Great job! You passed!\n";`) or the second (`cout << "I will see you again next semester\n";`). The first statement is often called the “true condition” and the second the “else condition”.

Observe how the `else` component of the IF statement aligns with the `if`. Also, both the true-condition and the else-condition are indented the same: three additional spaces. Finally, note that there is no semicolon after the Boolean expression nor after the `else`. This is because the entire IF-ELSE statement is considered a single C++ statement.

Sam's Corner



IF statements in C++ are compiled into JUMPZ (or one of many conditional jump) assembly statements. When the CPU encounters these statements, execution could either proceed to the next instruction or jump to the included address, depending on whether the Boolean expression is TRUE or not. Since CPUs like to look ahead in an effort to optimize processor performance, a decision must be made: is it more likely the Boolean expression evaluates to TRUE or FALSE? As a rule, all CPUs optimize on the TRUE condition. For this reason, there is a slight performance advantage for the TRUE condition to be the “most likely” of the two conditions.

Consider, for example, the above code. Since the vast majority of the students will achieve a grade of greater than 60%, the “Great job!” statement should be in the true-condition and the “next semester” statement should be in the else-condition. This will be slightly more efficient than the following code:

```
if (grade < 60)
    cout << "I will see you again next semester...\n";
else
    cout << "Great job! You passed!\n";
```

Example 1.6 – IF Statements

Demo

This example will demonstrate both types of IF statements: the Action/No-Action type and the Action-A/Action-B type.

Problem

Write a program to prompt the user for his GPA and display whether the value is in the valid range.

```
Please enter your GPA: 4.01
Your GPA is not in the valid range
```

The first part of the program is a function determining whether the GPA is within the acceptable range. It will take a `float` GPA as input and return a `bool`, whether the value is valid.

```

/*****
 * Demonstrating an Action-A/Action-B IF statement
 *****/
bool validGpa(float gpa)
{
    if (gpa > 4.0 || gpa < 0.0)           // Boolean expression
        return false;                   // True condition
    else
        return true;                     // False or Else condition
}

```

Note how two options are presented, the invalid range and the valid range.

The second part of the program displays an error message only in the case where GPA is outside the accepted range. This is an example of the Action/No-Action flavor.

```

/*****
 * Demonstrating an Action/No-Action IF statement
 *****/
int main()
{
    float gpa;

    // prompt for GPA
    cout << "Please enter your GPA: ";
    cin >> gpa;

    // give error message if invalid
    if (!validGpa(gpa))                 // Boolean expression
        cout << "Your GPA is not in a valid range\n"; // Action or Body of the IF

    return 0;
}

```

Challenge

As a challenge, see if you can modify the IF statement in `main()` to the Action-A/Action-B variety by displaying a message if the user has entered a valid GPA.

Another challenge would be to remove the IF statement from the `validGpa()` function and replace it with a simple boolean expression similar to what was done in Chapter 1.5

See Also

The complete solution is available at 1-6-ifStatements.cpp or:

```
/home/cs124/examples/1-6-ifStatements.cpp
```



Details

Anytime there is a place for a statement in C++, multiple statements can be added by using {}s. Similarly, whenever there is a place for a statement in C++, any statement can go there. For example, the body of an IF statement could contain another IF statement.

Compound Statements

Frequently we want to have more than one statement inside the body of the IF. Rather than duplicating the IF statement, we use {}s to surround all the statements going inside the body:

```
{
    if (!(classGrade >= 60))
    {
        classFail = true;
        classRetake = true;
        studentHappy = false;
    }
}
```

Each time an additional indentation is added, three more spaces are used. In this case, the IF statement is indented 3 spaces. Observe how the {}s align with the IF statement. The three assignment statements (such as `classFail = true;`) are indented an additional three spaces for a total of 6.

Nested Statements

Often we want to put an IF statement inside the body of another IF statement. This is called a nested statement because one statement is inside of (or part of) another:

```
{
    if (grade >= 90 && grade <= 100)
    {
        cout << "A";
        if (grade <= 93)
            cout << "-";
    }
}
```

Observe how the second COUT statement is indented 9 spaces, three more than the inner IF and six more than the outer IF. There really is no limit to the number of degrees of nesting you can use. However, when indentation gets too extreme (much more than 12), human readability of code often suffers.

Multi-Way

An IF statement can only differentiate between two options. However, often the program requires more than two options. This can be addressed by nesting IF statements:

```
{
    if (option == 1)
        cout << "Good!";
    else
    {
        if (option == 2)
            cout << "Better";
        else
            cout << "Best!";
    }
}
```

Observe how the inner {}s are actually not necessary. We only need to add {}s when more than one statement is used. Since a complete IF/ELSE statement (otherwise known as the Action-A/Action-B variant of an IF statement) is a single statement, {}s are not needed. Thus we could say:

```
{
    if (option == 1)
        cout << "Good!";
    else
        if (option == 2)
            cout << "Better";
        else
            cout << "Best!";
}
```

If we just change the spacing a little, we can re-arrange the code to a much more readable:

```
{
    if (option == 1)
        cout << "Good!";
    else if (option == 2)
        cout << "Better";
    else
        cout << "Best!";
}
```

This is the preferred style for a multi-way IF. Technically speaking, we can achieve a multi-way IF without resorting to ELSE statements.



Sue's Tips

Be careful and deliberate in the order in which the IF statements are arranged in multi-way IFs. Not only may a bug exist if they are in the incorrect order, but there may be performance implications as well. Make sure to put the most-likely or common cases at the top and the less-likely ones at the bottom.

Consider the following code:

```
{
    if (numberGrade >= 90 && numberGrade <= 100)
        letterGrade = 'A';

    if (numberGrade >= 80 && numberGrade < 90)
        letterGrade = 'B';

    if (numberGrade >= 70 && numberGrade < 80)
        letterGrade = 'C';

    if (numberGrade >= 60 && numberGrade < 70)
        letterGrade = 'D';

    if (numberGrade < 60)
        letterGrade = 'F';
}
```

Observe how each of the five IF statements stands on its own. This means that, every time the code is executed, each IF statement's Boolean expression will need to be evaluated. Also note how much of the Boolean

expressions are redundant. This statement has exactly the same descriptive power as the following multi-way IF:

```
{
    if (numberGrade >= 90)
        letterGrade = 'A';

    else if (numberGrade >= 80)
        letterGrade = 'B';

    else if (numberGrade >= 70)
        letterGrade = 'C';

    else if (numberGrade >= 60)
        letterGrade = 'D';

    else
        letterGrade = 'F';
}
```

Not only is this code much easier to read (simpler Boolean expressions) and less bug-prone (there is no redundancy), it is also much more efficient. Consider the case where `numberGrade == 93`. In this case, the first Boolean expression will evaluate to `true` and the body of the first IF statement will be executed. Since the entire rest of the multi-way IF is part of the ELSE condition of the first IF statement, it will all be skipped. Thus, far less code will be executed.

Pitfalls

The C++ language was designed to be as efficient and high-performance as possible. In other words, it was designed to facilitate making an efficient compiler so the resulting machine language executes quickly on the CPU. The C++ language was not designed to be easy to learn or easy to write code. Modern derivatives of C++ such as Java and C# were designed with that in mind. Taking this point into account, C++ programmers should always be on the look-out for various pitfalls that beset the language.

Pitfall: = instead of ==

Algebra treats the equals sign as a statement of equivalence, much like C++ treats the double equals sign. It is therefore common to mistakenly use a single equals when a double is needed:

```
{
    bool fail = false;
    if (fail = true)                // PITFALL: Assignment = used instead of ==
        cout << "You failed!\n";
}
```

In this statement, it may look like the program will display a message if the user has failed the class. Since the first statement sets `fail` to `false`, we will not execute the `cout` in the body of the IF. Closer inspection, however, will reveal that we are not *comparing* `fail` with `true`. Instead we are *setting* `fail` to `true`. Thus, the variable will change and the Boolean expression will evaluate to `true`.

Pitfall: Extra semicolon

Remember that the semicolon signifies the end of a statement. The end of an IF statement is the end of the statement inside of the body of the IF. Thus, if there is a semicolon after the Boolean expression, we are signifying that there is no body in the IF!

```
{
    if (false);                // PITFALL: Extra semicolon signifies empty body
        cout << "False!\n";
}
```

Because of the semicolon after the (false), the above code is equivalent to:

```
{
    if (false)                // If you mean to have an empty body,
        ;                    // then put it on its own line like this.
    cout << "False!\n";
}
```

In other words, the Boolean expression is ignored and the body of the IF is always executed!



Sue's Tips

Train your eye to look for extra semicolons on IF statements. They should never be there! If you meant to have an empty body (and you shouldn't!), then put the semicolon on its own line so the reader of the code knows it was intentional.

If the code editor tries to out-dent the body of your IF statement, pay attention: it is trying to tell you something important! The editor knows about semicolons and IF statements and is not fooled by this pitfall.

Pitfall: Missing {}s

In order to use a compound statement (more than one statement) in the body of an IF, it is necessary to surround the statements with {}s. A common mistake is to forget the {}s and just indent the statements. C++ ignores indentations (they are just used for human readability; the compiler throws away all white-spaces during the lexing process) and will not know the statements need to be in the body:

```
{
    if (classGrade < 60)
        classFail = true;
        classRetake = true;          // PITFALL: Missing {}s around the body of the IF
        studentHappy = false;
}
```

This is exactly the same as:

```
{
    if (classGrade < 60)
        classFail = true;
    classRetake = true;
    studentHappy = false;
}
```

Observe how only the first statement (classFail = true;) is part of the IF.

Example 1.6 – Overtime

Demo	This example will demonstrate how to send data to a function and receive data from a function. This follows the “Processing” model of information flow through a function.
Problem	Write a program to compute the hourly wage taking into account time-and-a-half overtime. In other words, if more than 40 hours are worked, then any additional hour benefits from a 50% increase in the wage. This can be solved only after the program makes a decision: are we using the regular formula (<code>hourlyWage * hoursWorked</code>) or the more complex overtime formula. What is your hourly wage? <u>10</u> How many hours did you work? <u>41</u> Pay: \$ 415.00
Solution	The function to compute pay taking the hourly wage and hours worked as input. <pre> /***** * COMPUTE PAY * Compute the user's pay using time-and-a-half * overtime. *****/ float computePay(float hourlyWage, float hoursWorked) { float pay; // regular rate if (hoursWorked < 40) pay = hoursWorked * hourlyWage; // regular rate // overtime rate else pay = (40.0 * hourlyWage) + // first 40 are normal... ((hoursWorked - 40.0) * (hourlyWage * 1.5)); // ...the balance overtime return pay; } </pre>
Challenge	Some companies credit employees with an hour of work each month even if they only worked a few minutes. In other words, there are four pay rates: no pay for those who did not work, a full hour's wage for those working less than an hour a week, regular wage, and the overtime wage. As a challenge, modify the above function to include this first-hour special case.
See Also	The complete solution is available at 1-6-overtime.cpp or: /home/cs124/examples/1-6-overtime.cpp 

Problem 1

What is the output?

```
void function(int b, int a)
{
    cout << "a == " << a << '\t'
        << "b == " << b << endl;
}

int main()
{
    int a = 10;
    int b = 20;

    cout << "a == " << a << '\t'
        << "b == " << b << endl;

    function(a, b);

    return 0;
}
```

Answer:

Please see page 62 for a hint.

Problem 2-8

What are the values of the following variables?:

```
{
    bool a = false && true || false && true;
    bool b = false || true && false || true;
    bool c = true && true && true && false;
    bool d = false || false || false || true;
    bool e = 100 > 90 > 80;
    bool f = 90 < 80 || 70;
    bool g = 10 + 2 - false;
}
```

Please see page 79 for a hint.

Problem 9-13

For each of the following, indicate where the parentheses goes to disambiguate the order of operations:

Raw expression	With parentheses
4 + 1 > 2 * 2	
a++ < !b	
a * b + c && d e	
3.1 * ! b > 7 * a++ == ++c + 2	
a < b < c < d	

Please see page 82 for a hint.

Problem 14

What is the output?

```
int subtract(int b, int a)
{
    return a - b;
}

int main()
{
    int c = subtract(4, 3);

    cout << c << endl;

    return 0;
}
```

Answer:

Please see page 62 for a hint.

Problem 15

Write a function to accept a number from the caller as a parameter and return whether the number is positive:

Please see page 87 for a hint.

Problem 16

What is the output?

```
{
    bool failedClass = false;
    int grade = 95;

    // pass or fail?
    if (grade < 60);
        failedClass = true;

    // output grade
    cout << grade << "%\n";

    // output status
    if (failedClass)
        cout << "You need to take "
            << "the class again\n";
}
```

Answer:

Please see page 92 for a hint.

Problem 17

What is the output?

```
{
    bool failedClass = false;
    int grade = 95;

    // pass or fail?
    if (grade = 60)
        failedClass = true;

    // output grade
    cout << grade << "%\n";

    // output status
    if (failedClass)
        cout << "You need to take "
            << "the class again\n";
}
```

Answer:

Please see page 91 for a hint.

Problem 18

What is the output when the user inputs the letter 'm'?

```
{
    char gender;

    // prompt for gender
    cout << "What is your gender? (m/f)";
    cin >> gender;

    // turn it into a bool
    bool isMale = true;
    if (gender == 'f');
        isMale = false;

    // output the result
    if (isMale)
        cout << "You are male!\n";
    else
        cout << "You are female!\n";
}
```

Answer:

Please see page 92 for a hint.

Problem 19

What is the output when the user inputs the number 5?

```
{
    int number;

    // prompt for number
    cout << "number? ";
    cin >> number;

    // crazy math
    if (number = 0)
        number += 2;

    // output
    cout << number << endl;
}
```

Answer:

Please see page 91 for a hint.

Assignment 1.6

Write a function (`computeTax()`) to determine which tax bracket a user is in. The tax tables are:

If taxable income is over--	But not over--	Bracket
\$0	\$15,100	10%
\$15,100	\$61,300	15%
\$61,300	\$123,700	25%
\$123,700	\$188,450	28%
\$188,450	\$336,550	33%
\$336,550	no limit	35%

The yearly income is passed as a parameter to the function. The function returns the percentage bracket that the user's income falls in. The return value from the function will be an integer (10, 15, 25, 28, 33, or 35).

Next write `main()` so that it prompts the user for his or her income and accepts the result from the `computeTax()` function and displays the result to the screen with a “%” after the number.

Examples

Three examples. The user input is in underline.

Example 1: 28%

Income: 150000
Your tax bracket is 28%

Example 2: 35%

Income: 1000000
Your tax bracket is 35%

Example 3: 10%

Income: 5
Your tax bracket is 10%

Assignment

The test bed is available at:

testBed cs124/assign16 assignment16.cpp

Don't forget to submit your assignment with the name “Assignment 16” in the header.

Please see page 89 for a hint.